Client-Server Architecture（客戶端-伺服器架構）

什麼是 Client-Server Architecture？

Client-Server Architecture 是現代網路應用最基本的架構模式。幾乎所有你在系統設計面試中遇到的系統，都建立在這個模型上。

核心概念非常簡單：

- **Client（客戶端）**：發起請求的一方。例如瀏覽器、手機 App、或另一個服務。
- **Server（伺服器）**：接收請求、處理邏輯、回傳結果的一方。

兩者透過網路溝通，遵循 **Request-Response（請求-回應）** 模型：客戶端發送請求，伺服器處理後回傳回應。

```
┌──────────┐    Request     ┌──────────┐
|          |  ──────────►   |          |
|  Client  |                |  Server  |
|          |  ◄──────────   |          |
└──────────┘    Response    └──────────┘
```

一個日常的例子：你打開瀏覽器輸入網址，瀏覽器（Client）向網站的伺服器（Server）發送 HTTP 請求，伺服器處理後回傳網頁內容，瀏覽器再把它渲染出來。

為什麼重要？

理解 Client-Server 是理解系統設計的起點，因為：

1. **職責分離**：Client 負責展示和互動，Server 負責商業邏輯和資料處理。這種分離讓系統更容易維護和擴展。
2. **集中管理**：資料和邏輯集中在 Server 端，方便更新、維護和安全管控。Client 不需要知道資料怎麼存、邏輯怎麼跑。
3. **多客戶端支援**：同一個 Server 可以同時服務網頁、手機 App、第三方 API 等不同類型的 Client。
4. **擴展基礎**：當流量增加時，我們可以對 Server 做垂直或水平擴展（見 `scalability.md`），而不需要改動 Client。

buildmoat.org

Client 和 Server 的角色

Client（客戶端）

Client 是使用者與系統互動的介面。常見的 Client 包括：

- **瀏覽器**（Chrome、Safari）：透過 HTTP/HTTPS 請求網頁和 API
- **手機 App**（iOS、Android）：透過 API 與後端溝通
- **桌面應用程式**：例如 Slack、VS Code
- **其他服務**：在微服務架構中，一個服務可以是另一個服務的 Client

Client 的典型職責：

- 呈現使用者介面（UI）
- 收集使用者輸入
- 向 Server 發送請求
- 接收並展示 Server 的回應

Server（伺服器）

Server 是處理請求的後端系統。它通常包含：

- **Web Server**：接收 HTTP 請求（例如 Nginx、Apache）
- **Application Server**：執行商業邏輯（例如 Node.js、Django、Spring Boot）
- **Database**：儲存和查詢資料（例如 PostgreSQL、MongoDB）

Server 的典型職責：

- 驗證和授權請求（Authentication & Authorization）
- 執行商業邏輯
- 讀寫資料庫
- 回傳結果給 Client

在實際的系統設計中，「Server」通常不是一台機器，而是由多台伺服器組成的叢集，前面有 Load Balancer 分配流量（見 `networking.md` 的負載平衡章節）。

Thin Client vs Thick Client

Client 可以承擔不同程度的邏輯處理：

Thin Client（瘦客戶端）：大部分邏輯在 Server 端，Client 只負責顯示。

buildnoat.org

- 例子：傳統的伺服器端渲染網頁（Server-Side Rendering），Server 產生完整的 HTML，瀏覽器只負責渲染。
- 優點：Client 輕量、易維護、安全性高（邏輯不暴露在前端）。
- 缺點：每次互動都需要向 Server 請求，使用者體驗可能較慢。

Thick Client（胖客戶端）：Client 端承擔較多邏輯，Server 主要提供資料。

- 例子：React / Vue 等 SPA（Single Page Application），前端處理路由、狀態管理、UI 邏輯，只向 Server 請求資料（API）。
- 優點：使用者體驗流暢，減少 Server 負擔。
- 缺點：前端程式碼複雜，部分邏輯暴露在 Client 端。

> **面試提示**：現代系統設計通常假設 Thick Client（SPA 或手機 App）+ RESTful API 的架構。你的設計重點通常在 Server 端的架構，但要知道 Client 端的選擇會影響 API 設計。

Client-Server vs Peer-to-Peer（P2P）

並非所有系統都使用 Client-Server 架構。另一種模型是 **Peer-to-Peer（P2P，點對點）**：

```
Client-Server:                    Peer-to-Peer:

    ┌──────┐                      ┌──────┐
    |Server|                      |Peer A|◄────►┌──────┐
    └──┬───┘                      └──┬───┘      |Peer C|
       |                             |          └──────┘
   ┌───┴───┐                         |
   |       |                      ┌──┴───┐
┌──┴───┐ ┌──┴───┐                  |Peer B|
| C1   | | C2   |                  └──────┘
└──────┘ └──────┘
```

特性	Client-Server	Peer-to-Peer
通訊方式	Client 向 Server 請求	節點之間直接溝通
集中控制	有（Server 控制一切）	無（去中心化）
擴展方式	擴展 Server	加入更多 Peer

buildmoat.org

特性	Client-Server	Peer-to-Peer
典型應用	幾乎所有 Web 應用	視訊通話（WebRTC）、檔案分享（BitTorrent）

> **面試提示**：系統設計面試中，99% 的問題都是 Client-Server 架構。只有在明確涉及視訊/音訊通話時，才需要考慮 P2P（WebRTC）。詳見 `networking.md` 的 WebRTC 章節。

在系統設計面試中的應用

在面試中畫系統架構圖時，Client-Server 是你的起點：

1. **從 Client 開始**：使用者透過什麼介面使用系統？（瀏覽器？App？）
2. **加入 Server**：處理請求的後端服務是什麼？
3. **定義 API**：Client 和 Server 之間怎麼溝通？（REST？GraphQL？WebSocket？）
4. **加入 Database**：Server 的資料存在哪裡？
5. **擴展**：流量大了怎麼辦？加 Load Balancer、Cache、CDN……

這就是為什麼 Client-Server 是基礎中的基礎——它是你畫出的第一條線，後續所有的設計都是在這個框架上展開的。

自我測驗

Q1：Client 和 Server 的核心職責分別是什麼？

> - **Client**：發起請求、呈現 UI、收集使用者輸入、展示回應。
> - **Server**：接收請求、驗證授權、執行商業邏輯、讀寫資料庫、回傳結果。

Q2：為什麼 Client-Server 架構中的職責分離很重要？

> 職責分離讓系統更容易維護和擴展。Server 端可以獨立更新邏輯和資料結構，而不影響 Client；同一個 Server 也可以服務多種不同的 Client（網頁、App、第三方）。當流量增加時，可以只擴展 Server 端而不需要改動 Client。

Q3：Thin Client 和 Thick Client 的主要差異是什麼？現代系統設計通常偏向哪一種？

> - **Thin Client**：邏輯在 Server 端，Client 只負責渲染（如傳統 SSR 網頁）。
> - **Thick Client**：Client 承擔較多邏輯，Server 主要提供資料 API（如 React SPA、手機 App）。
>
> 現代系統設計通常偏向 **Thick Client + RESTful API** 的架構。

buildmoat.org

Q4：什麼時候應該考慮 P2P 而不是 Client-Server？

> 當需要客戶端之間的**直接**、**低延遲通訊**時，例如視訊/音訊通話（WebRTC）。在系統設計面試中，除非題目明確涉及即時音視訊通話，否則預設使用 Client-Server 架構。

buildmoat.org